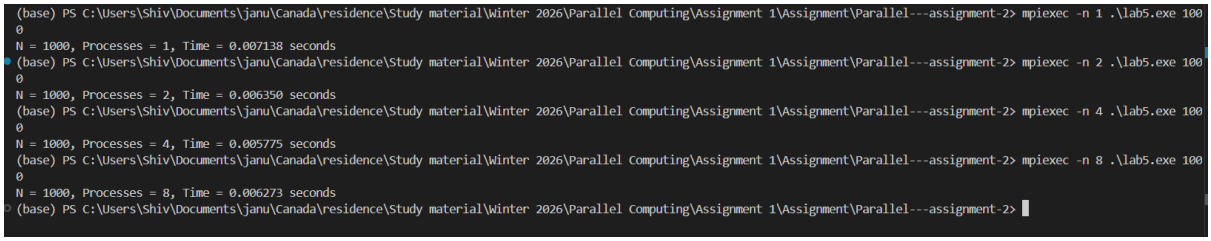
Lab 5: Performance Report

Name: Jayashree Janani Purushothaman

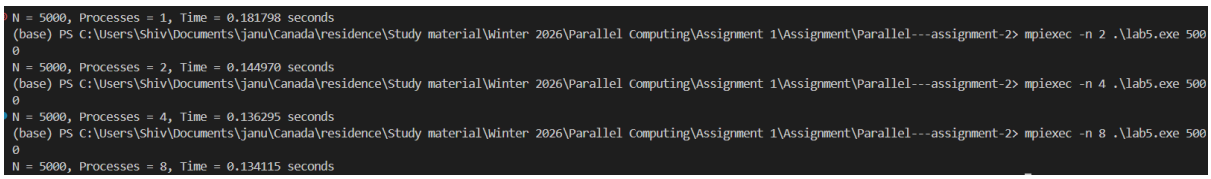
Student#: 3772545

Matrix Size(N)	P = 1	P=2	P=4	P=8
1000	0.007138	0.006350	0.005775	0.006273
5000	0.181798	0.144970	0.136295	0.134115
10000	0.757113	0.556174	0.548853	0.582890
20000	2.989316	2.059529	2.002465	1.782628

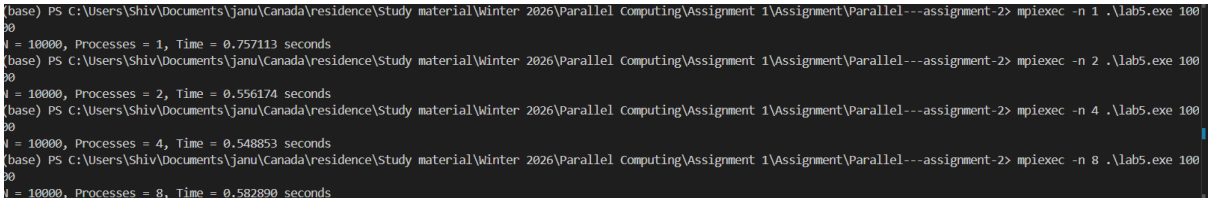
Matrix Size = 1000



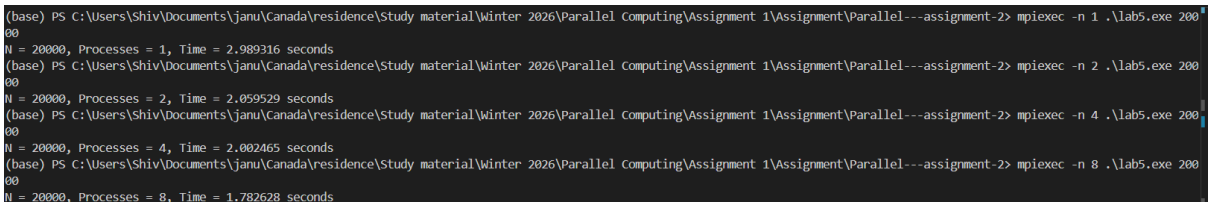
Matrix Size = 5000



Matrix Size = 10000



Matrix Size = 20000



Speed up Analysis

Matrix Size(N)	S(2)	S(4)	S(8)
1000	1.124094	1.236017	1.137893
5000	1.254039	1.333857	1.355538
10000	1.361288	1.379446	1.298895
20000	1.451456	1.492818	1.676915

1. Does the program achieve linear speedup?

No, the program does not achieve linear speedup. If it were linear, the speedup would be close to the number of processes used, such as 2 for 2 processes, 4 for 4 processes, and 8 for 8 processes. However, in my results, the speedups are much lower than that. For example, for $N = 20000$, the speedup at 8 processes is about 1.68, not 8. This shows that the program benefits from parallelism, but the improvement is not perfectly proportional to the number of processes.

2. What factors limit scalability?

The main factors that limit scalability are communication overhead and synchronization cost. In this program, processes need to communicate using MPI_Scatter, MPI_Bcast, and MPI_Gather, and that communication takes time. As the number of processes increases, the communication overhead also increases. Another factor is memory access cost, since the matrix is large and rank 0 has to manage the full matrix. Because of these reasons, the speedup is limited and does not grow linearly.

3. Why does increasing matrix size improve speedup?

Increasing the matrix size improves speedup because the amount of computation becomes larger compared to the communication cost. For small matrix sizes like $N = 1000$, the program spends a noticeable amount of time on communication, so the speedup is low. For larger matrix sizes like $N = 20000$, each process has more computation to do, so the parallel execution becomes more useful. This is why the speedup generally improves as Nincreases in my results.

4. Why can very large matrix sizes cause program failure?

Very large matrix sizes can cause program failure because the matrix requires a lot of memory. The matrix is stored as double values, and memory usage grows very quickly as N^2. When Nbecomes very large, the system may not have enough memory to allocate the full matrix, especially on rank 0, which stores the complete matrix. This can lead to memory allocation failure or the program crashing.

5. Small conclusion based on your results

From my results, I can see that the MPI implementation improves performance as the matrix size increases, but the speedup is not linear. The best speedup in my experiment was for $N = 20000$with 8 processes, where the speedup was about 1.68. This shows that parallel matrix-vector multiplication gives better results for larger workloads, but communication and memory overhead still reduce the overall efficiency.